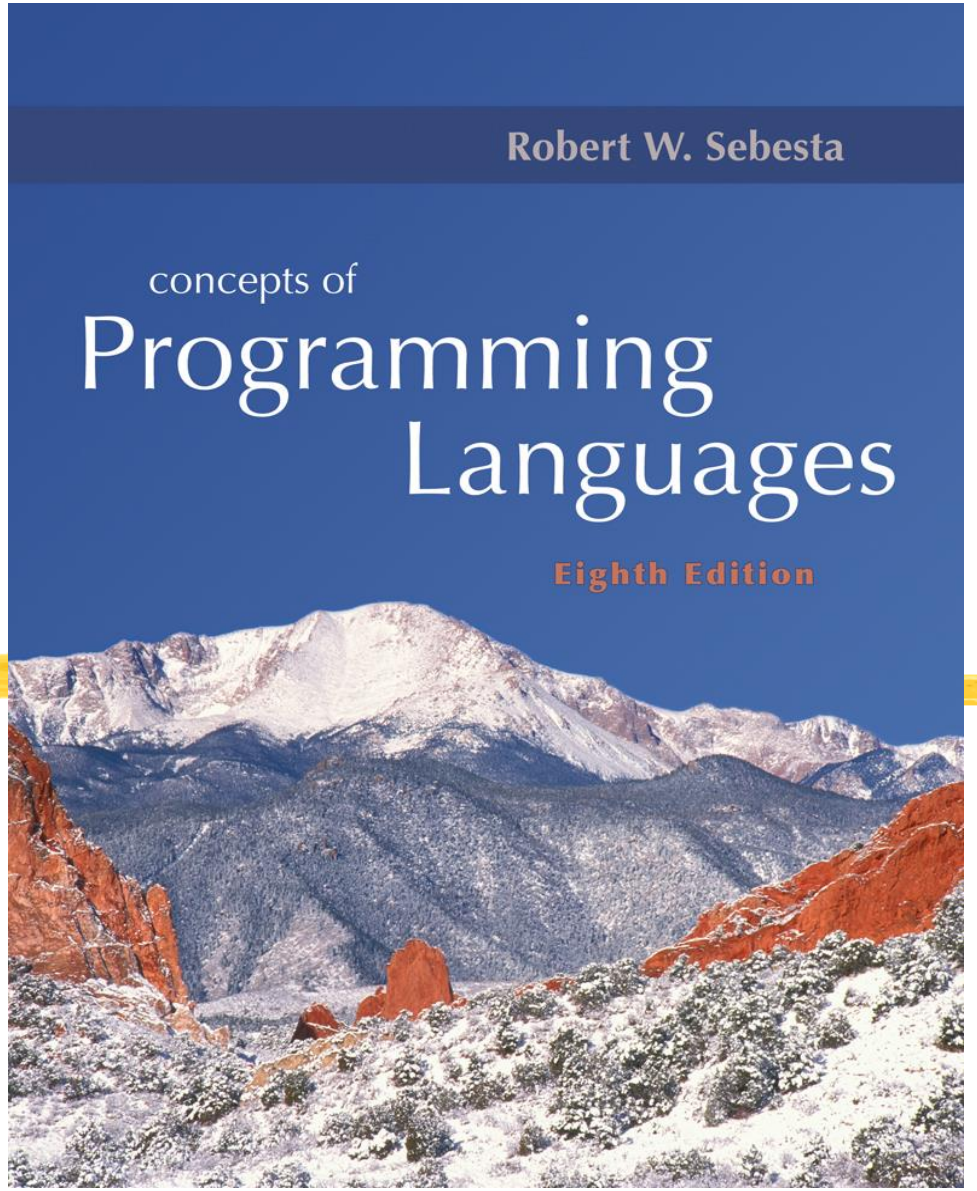
CPCS 301 Concepts of Programming Languages

Preliminaries

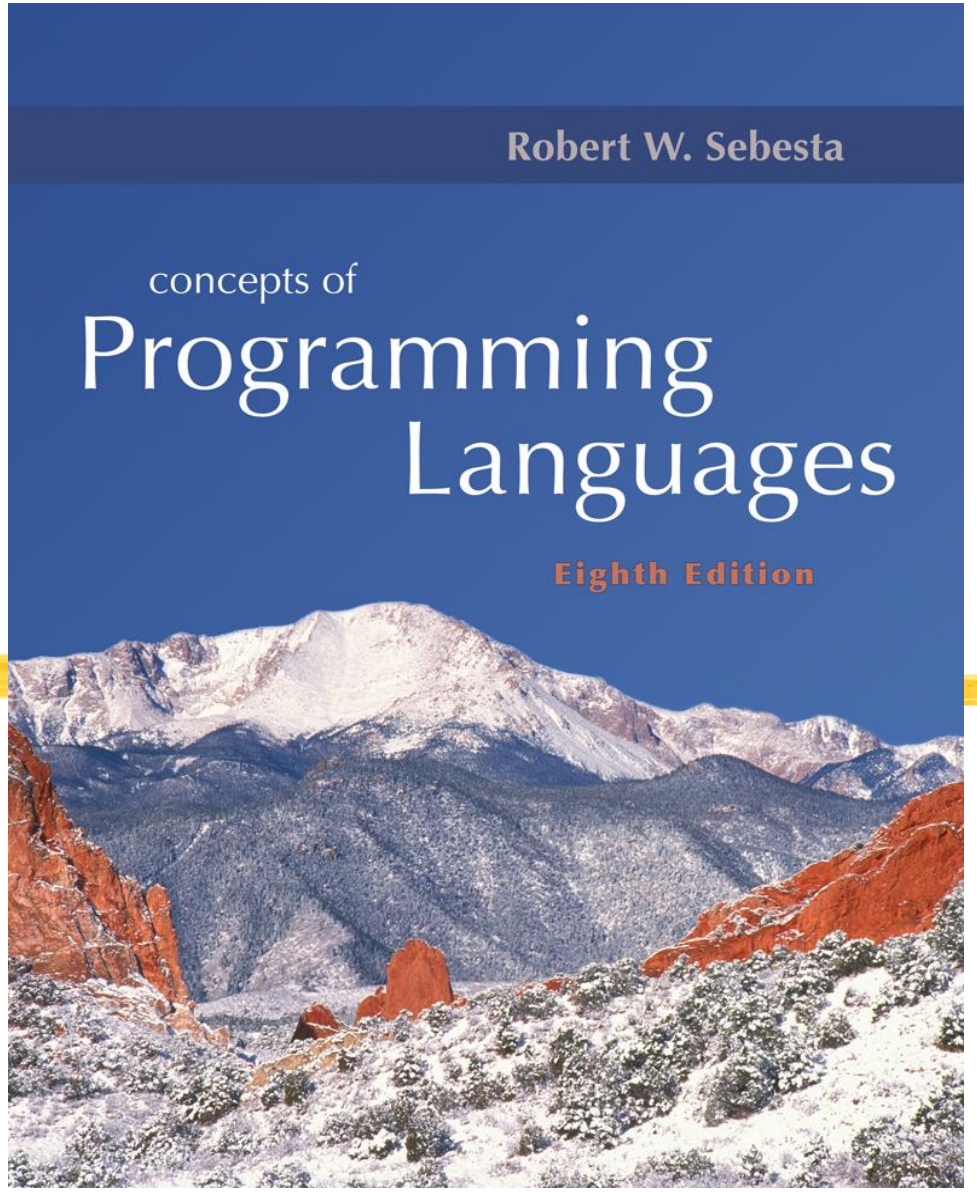


Dr. Asaad Ahmed
Department of Computer Science
Faculty of Computing and
Information Technology
King Abdulaziz University

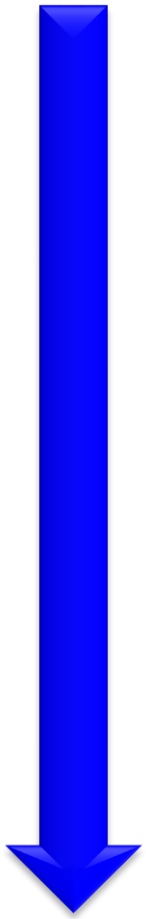
(Slides are adopted from *Concepts of Programming Languages*, R.W. Sebesta; *Modern Programming Languages: A Practical Introduction,* A.B. Webber)

Chapter 3

Describing Syntax and Semantics



Roadmap



Ch. 1	Classification of languages What make a "good" language?
Ch. 2	Evolution of languages
Ch. 3	How to define languages?
Ch. 4	How to compile and translate programs?
Ch. 5	Variables in languages
Ch. 7	Statements and program constructs in languages
Ch. 15	Functional and logic languages

Chapter 3 Topics

- Introduction
- The General Problem of Describing Syntax
- Formal Methods of Describing Syntax
- Attribute Grammars
- Describing the Meanings of Programs: Dynamic Semantics

Introduction

- **Syntax:** the form or structure of the expressions, statements, and program units
- **Semantics:** the meaning of the expressions, statements, and program units
- **Syntax** and **semantics** provide a language's definition
  - Users of a language definition
    - Other language designers
    - Implementers
    - Programmers (the users of the language)

Describing Syntax and Semantics

- Syntax is defined using some kind of rules
  - Specifying how statements, declarations, and other language constructs are written
- Semantics is more complex and involved. It is harder to define, e.g., natural language doc.
- Example: **if** statement
  - Syntax: **if (<expr>) <statement>**
  - Semantics: if **<expr>** is true, execute **<statement>**
- Detecting syntax error is easier, semantics error is much harder

The General Problem of Describing Syntax: Terminology

- A *language* is a set of sentences
- A *sentence* is a string of characters over some alphabet
  - The meaning of a "sentence" is very general. In English, it may be an English sentence, a paragraph, or all the text in a book, or hundreds of books, …
- A *lexeme* is the lowest level syntactic unit of a language (e.g., `*`, `sum`, `begin`)
- A *token* is a category of lexemes (e.g., identifier)

A Sentence in C Language

- Every C program, if can be compiled properly, is a sentence of the C language
  - No matter whether it is "hello world" or a program with several million lines of code
- The "Hello World" program is a sentence in C

```
main()
{  printf("hello, world!\n"); }
```

A Sentence in C Language

- What about its alphabet?
  - For illustration purpose, let us define the alphabet as

    a → identifier b → string c→'('

    d→')' e→'{' f→'}' g→';'
  - So, symbolically "Hello World" program can be represented by the sentence: acdeacbdgf where "main" and "printf" are identifiers and "hello, world!\n" is a string

```
main()
{  printf("hello, world!\n"); }
```

=> acdeacbdgf

Sentence and Language

- So, we say that acdeacbdgf is a sentence of (or, in) the C language, because it represents a legal program in C
  - Note: "legal" means syntactically correct
- How about the sentence **acdeacbdf**?
  - It represents the following program:

```
main()
{  printf("hello, world!\n") }
```

  - Compiler will say there is a syntax error
  - In essence, it says the sentence acdeacbdf is not in C language

So, What a C Compiler Does?

- **Frontend**: check whether the program is "a sentence of the C language"
  - Lexical analysis: translate C code into corresponding "sentence" (intermediate representation, IR) → Ch. 4
  - Syntax analysis: check whether the sentence is a sentence in C → Ch. 4
    - Not much about what it means → semantics
- **Backend**: translate from "sentence" (IR) into object code
  - Local and global optimization
  - Code generation: register and storage allocation, …

Formal Definition of Languages

- **Recognizers**
  - A recognition device reads input strings of the language and decides whether the input strings belong to the language
  - Example: syntax analysis part of a compiler
  - Detailed discussion in Chapter 4

- **Generators**
  - A device that generates sentences of a language
  - One can determine if the syntax of a particular sentence is correct by comparing it to the structure of the generator

Formal Definition of Languages

- The syntax of a language can be defined by a set of syntax rules
- The syntax rules of a language specify which sentences are in the language, i.e., which sentences are legal sentences of the language

Syntax Rules

- Consider a special language X containing sentences such as

  Jedda is in KSA.

  Jedda belongs to KSA.

- A general rule of the sentences in X may be

  A sentence consists of a noun followed by a verb, followed by a preposition, and followed by a noun,

  where a noun is a *place*

  a verb can be "is" or "belongs" and

  a preposition can be "in" or "to"

Syntax Rules

A hierarchical structure of language

- A more concise representation:

  <sentence> → <noun> <verb> <preposition> <noun>

  <noun> → *place*

  <verb> → "is" | "belongs"

  <preposition> → "in" | "to"

- With these rules, we can generate followings:

  Jedda is in KSA

  Jedda belongs to KSA

- They are all in language X
  - Its alphabet includes "is", "belongs", "in", "to", *place*

Checking Syntax of a Sentence

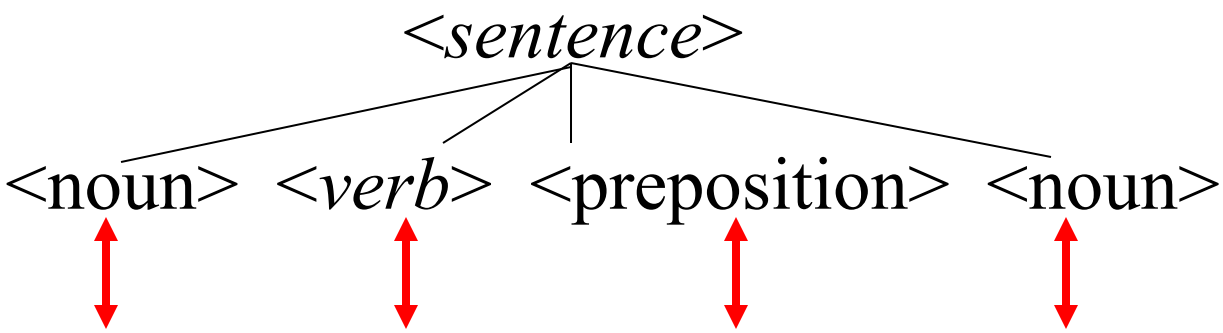
- How to check if the following sentence is in the language X?

  Jedda belongs in KSA

- Idea: check if you can generate that sentence
  → This is called *parsing*

- How?
  Try to match the input sentence with the structure of the language

Matching the Language Structure



So, the sentence is in the language X!

Jedda belongs in KSA

The above structure is called a parse tree

Formal Methods of Describing Syntax

- **Backus-Naur Form and Context-Free Grammars**
  - Most widely known method for describing programming language syntax
- **Extended BNF**
  - Improves readability and writability of BNF
- **Grammars and Recognizers**

BNF and Context-Free Grammars

- Context-Free Grammars
  - Developed by Noam Chomsky in the mid-1950s
  - Language generators, meant to describe the syntax of natural languages
  - Define a class of languages called context-free languages

BNF and Context-Free Grammars

- Backus-Naur Form (1959)
  - Invented by John Backus to describe Algol 58
  - BNF is equivalent to context-free grammars
  - BNF is a *metalanguage* used to describe another language
  - In BNF, abstractions are used to represent classes of syntactic structures--they act like syntactic variables (also called *nonterminal symbols*)

BNF Fundamentals

- Non-terminals: BNF abstractions,
  - e.g., <sentence>, <verb>
  - There is a special non-terminal called the *start* symbol, e.g., <sentence>
- Terminals: lexemes and tokens
- Grammar: a collection of rules
  - Examples of BNF rules:

```
<ident_list> → <ident> | <ident>, <ident_list>
<if_stmt> → if <logic_expr> then <stmt>
```

BNF Terminologies

- *Tokens* and *lexemes* are smallest units of syntax
  - Lexemes appear literally in program text
- *Non-terminals* stand for larger pieces of syntax
  - Do NOT occur literally in program text
  - The grammar says how they can be expanded into strings of tokens or lexemes
- The *start* symbol is the particular non-terminal that forms the starting point of generating a sentence of the language

Context Free Grammar (CFG) (BNF)

- $\mathcal{G} = (\mathcal{T}, \mathcal{N}, \mathcal{P}, S)$
  - $\mathcal{T}$: set of terminals
  - $\mathcal{N}$: set of non-terminals
  - $\mathcal{P}$: set of production rules
  - S : start symbol and S $\in$ N

CFG Example

<stmt> → <single_stmt>
| begin <stmt_list> end

- $\mathcal{T}$ = {begin, end}
- $\mathcal{N}$ = {<stmt>,<single_stmt>,<stmt_list>}
- $\mathcal{P}$ = {<stmt> → <single_stmt>
| begin <stmt_list> end}

Or

- $\mathcal{P}$ = {<stmt> → <single_stmt> ,
<stmt> → begin <stmt_list> end}
- $\mathcal{S}$ = <stmt>

CFG (BNF) Formats

- <stmt> → <single_stmt>
  | begin <stmt_list> end
- <stmt> ::= <single_stmt>
  | begin <stmt_list> end
- stmt ::= single_stmt
  | begin stmt_list end
- **stmt : single_stmt**
  begin **stmt_list** end

STMT	**SINGLE_STMT**
	begin **STMT_LIST** end

BNF Rules

- A rule has a left-hand side (LHS) and a right-hand side (RHS)
  - consists of *terminal* and *nonterminal* symbols
  - LHS is a single non-terminal → context-free
  - RHS contains one or more terminals or non-terminals
  - A rule tells how LHS can be replaced by RHS, or how RHS is grouped together to form a larger syntactic unit (LHS) → traversing the parse tree up and down
  - A nonterminal can have more than one RHS

BNF Rules

- A grammar is a finite nonempty set of rules
- An abstraction (or nonterminal symbol) can have more than one RHS

```
<stmt> → <single_stmt>
       | begin <stmt_list> end
```

Describing Lists

- Syntactic lists are described using recursion

```
<ident_list> → <ident>
             | <ident>, <ident_list>
```

An Example Grammar

<program> → <stmts>

<stmts> → <stmt> | <stmt> ; <stmts>

<stmt> → <var> = <expr>

<var> → a | b | c | d

<expr> → <term> + <term> | <term> - <term>

<term> → <var> | <const>

<const> → 1|2|3|4|5|6|7|9|0

Derivation

- A *derivation* is a repeated application of rules, starting with the start symbol and ending with a sentence (all terminal symbols)

Derivation

- Every string of symbols in the derivation is a sentential form
- A sentence is a sentential form that has only terminal symbols
- A leftmost derivation is one in which the leftmost nonterminal in each sentential form is the one that is expanded
- A derivation may be neither leftmost nor rightmost

An Example Derivation

<program> => <stmts> => <stmt>

=> <var> = <expr> => a =<expr>

=> a = <term> + <term>

=> a = <var> + <term>

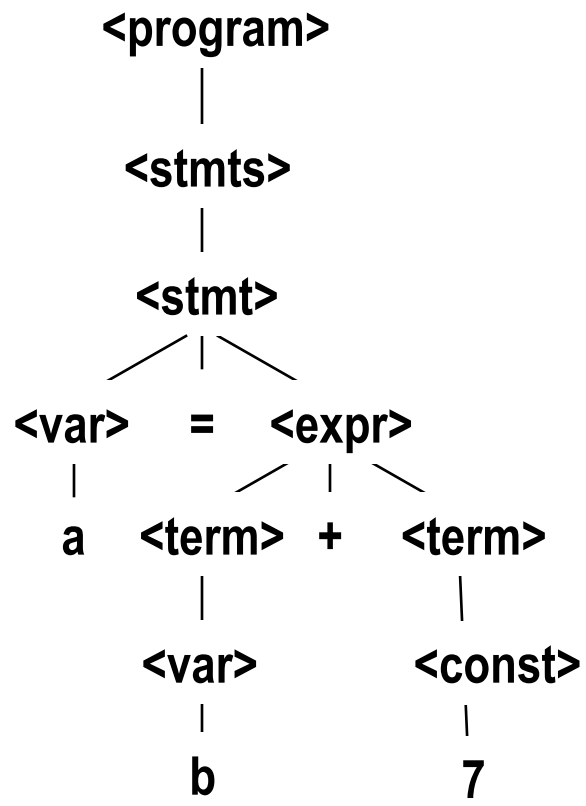
=> a = b + <term>

=> a = b + <const>

=> a = b + 7

Parse Tree

- A hierarchical representation of a derivation



a = b + 7

Grammar and Parse Tree

- The grammar can be viewed as a set of rules that say how to build a parse tree
- You put <S> at the root of the tree
- Add children to every non-terminal, *following any one of the rules for that non-terminal*
- Done when all the leaves are tokens
- Read off leaves from left to right—that is the string derived by the tree
  - e.g., in the case of C language, the leaves form the C program, despite it has millions of lines of code

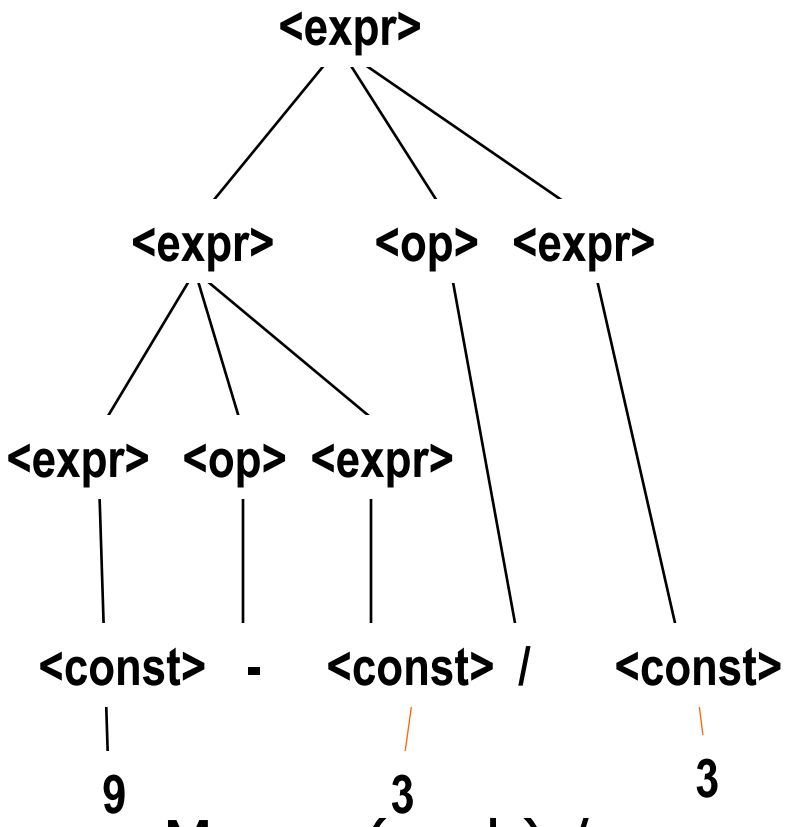
Ambiguity in Grammars

- A grammar is *ambiguous* if and only if it generates a sentential form that has two or more distinct parse trees
- Problem with ambiguity:
  - Consider the following grammar and the sentence **a-b/c**

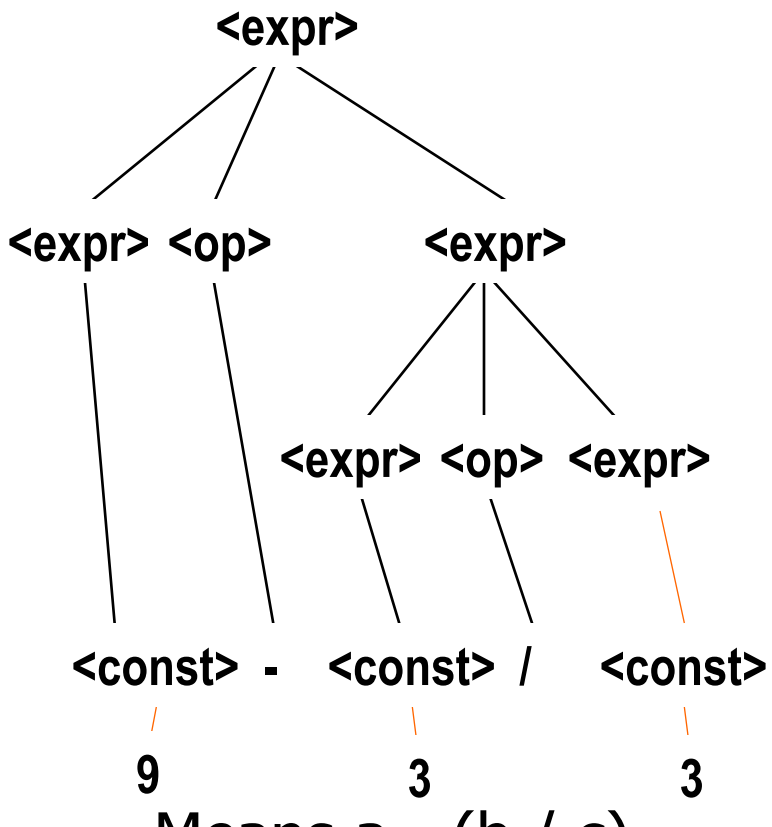
<expr> → <expr> <op> <expr> | <const>
<op> → / | -
<const> → 1|2|3|4|5|6|7|9|0

An Ambiguous Expression Grammar

<expr> → <expr> <op> <expr> | <const>
<op> → / | -
<const> → 1|2|3|4|5|6|7|9|0



Means (a - b) /c



Means a - (b / c)

Extended BNF

- Optional parts are placed in brackets [ ]

  <proc_call> -> <ident> [(<expr_list>)]

- Alternative parts of RHSs are placed inside parentheses and separated via vertical bars

  <term> → <term> (+|-) <const>

- Repetitions (0 or more) are placed inside braces { }

  <ident> → <letter> {<letter>|<digit>}

BNF and EBNF

- BNF

```
<expr> → <expr> + <term>
         | <expr> - <term>
         | <term>
<term> → <term> * <factor>
         | <term> / <factor>
         | <factor>
```

- EBNF

```
<expr> → <term> {(+ | -) <term>}
<term> → <factor> {(* | /) <factor>}
```

Attribute Grammars

- Context-free grammars (CFGs) cannot describe all of the syntax of programming languages
- Additions to CFGs to carry some semantic info along parse trees
- Primary value of attribute grammars (AGs)
  - Static semantics specification
  - Compiler design (static semantics checking)

Attribute Grammars : Definition

- An attribute grammar is a context-free grammar G = (S, N, T, P) with the following additions:
  - For each grammar symbol *x* there is a set *A(x)* of attribute values
  - Each rule has a set of functions that define certain attributes of the nonterminals in the rule
  - Each rule has a (possibly empty) set of predicates to check for attribute consistency

Attribute Grammars: Definition

- Let $X_0 \rightarrow X_1 \ldots X_n$ be a rule
- Functions of the form $S(X_0) = f(A(X_1), \ldots, A(X_n))$ define *synthesized attributes*
- Functions of the form $I(X_j) = f(A(X_0), \ldots, A(X_n))$, *for i <= j <= n*, define *inherited attributes*
- Initially, there are *intrinsic attributes* on the leaves

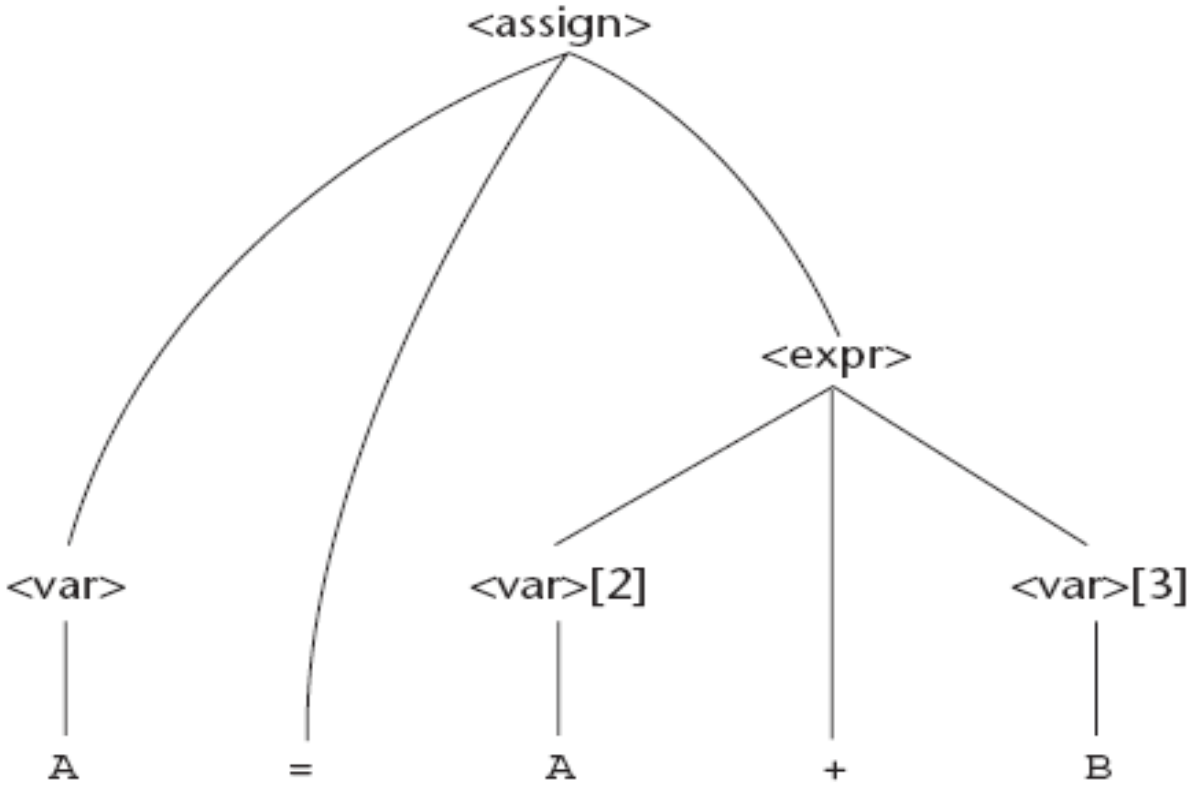
Attribute Grammars: Definition

An Attribute Grammar for Simple Assignment Statements

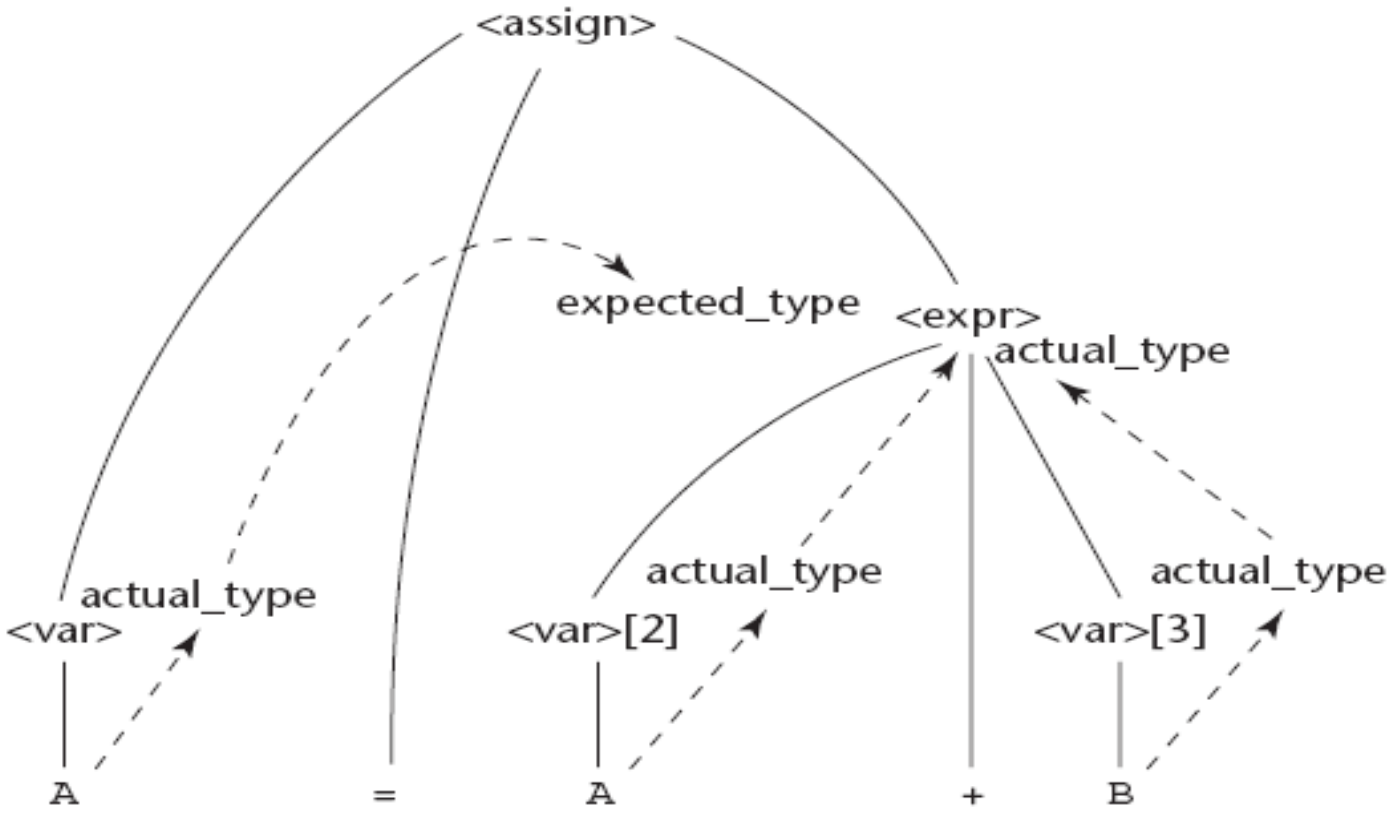
1. Syntax rule: <assign> → <var> = <expr>
   Semantic rule: <expr>.expected_type ← <var>.actual_type
2. Syntax rule: <expr> → <var>[2] + <var>[3]
   Semantic rule: <expr>.actual_type ←
   if (<var>[2].actual_type = int) and
   (<var>[3].actual_type = int)
   then int
   else real
   end if
   Predicate: <expr>.actual_type == <expr>.expected_type
3. Syntax rule: <expr> → <var>
   Semantic rule: <expr>.actual_type ← <var>.actual_type
   Predicate: <expr>.actual_type == <expr>.expected_type
4. Syntax rule: <var> → A | B | C
   Semantic rule: <var>.actual_type ← look-up (<var>.string)

The look-up function looks up a given variable name in the symbol table and returns the variable's type.

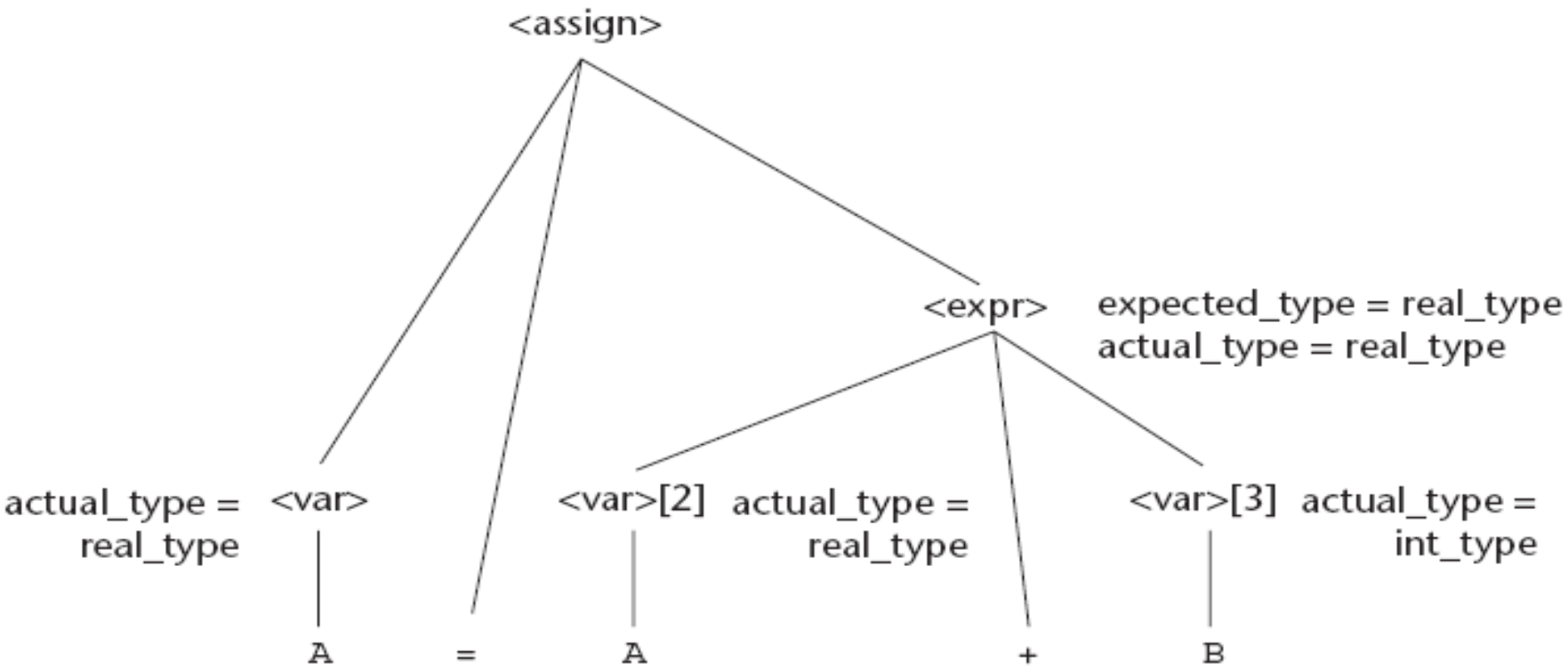
Attribute Grammars: Definition



Attribute Grammars: Definition



Attribute Grammars: Definition



Attribute Grammars: An Example

- Syntax

  <assign> -> <var> = <expr>

  <expr> -> <var> + <var> | <var>

  <var> -> A | B | C

- actual_type: synthesized for <var> and <expr>
- expected_type: inherited for <expr>

Attribute Grammar (continued)

- Syntax rule: <expr> → <var>[1] + <var>[2]
  Semantic rules:
  <expr>.actual_type ← <var>[1].actual_type
  Predicate:
  <var>[1].actual_type == <var>[2].actual_type
  <expr>.expected_type == <expr>.actual_type

- Syntax rule: <var> → id
  Semantic rule:
  <var>.actual_type ← lookup (<var>.string)

Attribute Grammars (continued)

- How are attribute values computed?
  - If all attributes were inherited, the tree could be decorated in top-down order.
  - If all attributes were synthesized, the tree could be decorated in bottom-up order.
  - In many cases, both kinds of attributes are used, and it is some combination of top-down and bottom-up that must be used.

Pattern Matching (Regular Expressions)

- A **regular expression** (**regex** for short) is a special text string for describing a search **pattern**
- You can think of **regular expressions** as wildcards
- A regular expression is written in a formal language that can be interpreted by a regular expression processor
- such as "*.txt" to find all text files in a file manager

Regular Expressions (Cont')

Metacharacter	Description	Examples
character	Any literal letter, number, or punctuation character (other than those that follow) matches itself.	apple matches apple.
(pattern)	Patterns can be grouped together using parentheses so that they can be treated as a unit.	see following
.	Match a single character (except linefeed).	s.t matches sat, sit, sQt, s3t, s&t, s t,...
?	Match zero or one of the previous character/expression. (When immediately following ?, +, *, or {min,max} it prevents the expression from using "greedy" evaluation.)	colou?r matches color, colour
+	Match one or more of the previous character/expression.	a+rgh! matches argh!, aargh!, aaargh!,...
*	Match zero or more of the previous character/expression.	b(an)*a matches ba, bana, banana, bananana,...
{number}	Match exactly number copies of the previous character/expression.	.o{2}n matches noon, moon, loon,...

Regular Expressions (Cont')

Metacharacter	Description	Examples
{min,max}	Match between min and max copies (inclusive) of the previous character/expression.	kabo{2,4}m matches kaboom, kabooom, kaboooom.
[set]	Match a single character in set (list and/or range). Most characters that have special meanings in regular expressions do not have to be backslash-escaped in character sets.	J[aio]b matches Jab, Jib, Job [A-Z][0-9]{3} matches Canadian postal codes.
[^set]	Match a single character not in set (list and/or range). Most characters that have special meanings in regular expressions do not have to be backslash-escaped in character sets.	q[^u] matches very few English words (Iraqi? qoph? qintar?).
|	Match either expression that it separates.	(Mi|U)nix matches Minix and Unix
^	Match the start of a line.	^# matches lines that begin with #.
$	Match the end of a line.	^$ matches an empty line.

Regular Expressions (Cont')

Metacharacter	Description	Examples
\	Interpret the next metacharacter character literally, or introduce a special escaped combination (see following).	* matches a literal asterisk.
\n	Match a single newline (carriage return in Python) character.	Hello\nWorld matches Hello World.
\t	Match a single tab character.	Hello\tWorld matches Hello World.
\s	Match a single whitespace character.	Hello\s+World matches Hello World, Hello World, Hello World,...
\S	Match a single non-whitespace character.	\S\S\S matches AAA, The, 5-9,...
\d	Match a single digit character.	\d\d\d matches 123, 409, 982,...
\D	Match a single non-digit character.	\D\D matches It, as, &!,...

www.regex101.com

Axiomatic Semantics

- Based on formal logic (predicate calculus)
- Original purpose: formal program verification
- Axioms or inference rules are defined for each statement type in the language (to allow transformations of expressions to other expressions)
- The expressions are called *assertions*

Axiomatic Semantics (continued)

- An assertion before a statement (a *precondition*) states the relationships and constraints among variables that are true at that point in execution
- An assertion following a statement is a *postcondition*
- A *weakest precondition* is the least restrictive precondition that will guarantee the postcondition

Axiomatic Semantics Form

- Pre-, post form: `{P} statement {Q}`
- An example
  - `a = b + 1  {a > 1}`
  - One possible precondition: `{b > 10}`
  - Weakest precondition: `{b > 0}`

Program Proof Process

- The postcondition for the entire program is the desired result
  - Work back through the program to the first statement.
  - If the precondition on the first statement is the same as the program specification, the program is correct.

Program Proof Process

```
x = 2 * y - 3 {x > 25}
```

The precondition is computed as follows:

```
2 * y - 3 > 25
y > 14
```

Axiomatic Semantics: Axioms

- An axiom for assignment statements

  (x = E): $\{Q_{x->E}\}$ x = E {Q}

- The Rule of Consequence:

$$\frac{\{P\}\,S\,\{Q\},\ P' \Rightarrow P,\ Q \Rightarrow Q'}{\{P'\}\,S\,\{Q'\}}$$

$$\frac{\{x>3\}x = x-3\{x>0\},\ (x>5) \Rightarrow \{x>3\},\ (x>0) \Rightarrow (x>0)}{\{x>5\}x = x-3\{x>0\}}$$

Axiomatic Semantics: Axioms

- An inference rule for sequences

  {P1} S1 {P2}

  {P2} S2 {P3}

$$\frac{\{P1\}\,S1\,\{P2\},\{P2\}\,S2\,\{P3\}}{\{P1\}\,S1;S2\,\{P3\}}$$

Summary

- BNF and context-free grammars are equivalent meta-languages
  - Well-suited for describing the syntax of programming languages
- An attribute grammar is a descriptive formalism that can describe both the syntax and the semantics of a language
- Three primary methods of semantics description

- The attribute a of symbol X is denoted X.a

- We say that an attribute X.a is synthesized
  - if there is a grammar rule X ::= α and
  - X.a is defined in terms of the attributes of the elements of α.

- We say that X.a is inherited
  - if there is a rule Y ::= $\alpha X \beta$ and
  - X.a is defined in terms of the Y, α, and β